ספר לימוד והדרכה מעשית

MAUI Learning Book

המדריך המעשי לפיתוח אפליקציות מובייל ודסקטופ ב-.NET MAUI

פרויקט מלווה: LearnSqlite (.NET MAUI & SQLite)

ארכיטקטורה: MVVM, Service Layer & Repository Pattern

טכנולוגיות: C# 12, .NET 10, XAML, SQLite-net-pcl, xUnit

שנת מהדורה: 2026

ספר הלמידה וההדרכה: LearnSqlite — מארכיטקטורת MVVM ועד מסד נתונים SQLite

ברוכים הבאים לספר הלמידה המלווה את פרויקט **LearnSqlite** ב-.NET MAUI! מטרת הפרויקט היא להוביל תלמידים צעד-אחר-צעד מבניית אפליקציית **MVVM** תלת-מסכית נקייה העובדת מול נתוני דמה בזיכרון (**Mockup Service** בענף `master`), ועד להטמעת מסד נתונים מקומי אמיתי מבוסס **SQLite** בענף ההמשך — מבלי לשבור את קוד המסכים או ה-ViewModels.

---

1. מפת הדרכים הפדגוגית של הפרויקט (Project Roadmap)

הפרויקט בנוי בשני שלבים מרכזיים (ענפי Git נפרדים), כך שהתלמיד חווה בעצמו את הכוח האמיתי של הפרדת שכבות וממשקים (`Interfaces`):

שלב / ענף	תוצאה נראית לעין	קבצים מרכזיים	מושג מפתח
שלב א' (`master`)	אפליקציית 3 מסכים פעילה עם CRUD מלא בזיכרון	`HeroItem`, `IHeroService`, `MockHeroService`, 3 `ViewModels`, 3 `Views`	הפרדת שכבות, `INotifyPropertyChanged`, `ICommand`, ניווט `AppShell`
שלב ב' (`sqlite`)	שמירה קבועה בקובץ מסד נתונים מקומי במכשיר	`SqliteHeroService`, תכונות `[PrimaryKey]` במודל	`SQLiteAsyncConnection`, מיפוי ORM, החלפת שירות שקופה

> ⚠️ **למה מתחילים עם `MockHeroService` לפני שמתקינים SQLite?**
>
> כאשר מפרידים בין **ממשק השירות** (`IHeroService`) לבין **המימוש שלו** (`MockHeroService`), התלמיד בונה ובודק את כל מסכי האפליקציה, הטפסים, הפקודות והניווט בסביבה פשוטה ומהירה. לאחר שהאפליקציה עובדת ב-100%, המעבר ל-SQLite מתמקד אך ורק בשכבת הנתונים!

---

2. ארכיטקטורת השכבות בפרויקט הבסיס (`master`)

פרויקט הבסיס בנוי בארכיטקטורת **MVVM** מוקפדת, ללא הזרקת תלויות (DI) בשלב זה, כדי לשמור על שקיפות מלאה של זרימת האובייקטים עבור התלמיד:

TEXT

```
LearnSqlite/
├── Models/
│   └── HeroItem.cs              # מודל הנתונים (POCO) של גיבור-על
├── Services/
│   ├── IHeroService.cs          # חוזה א-סינכרוני לכל פעולות ה-CRUD
│   └── MockHeroService.cs       # מימוש בזיכרון עם 6 גיבורי-על ותמונות חיות
├── ViewModels/
```

```
│   ├── BaseViewModel.cs          # מימוש INotifyPropertyChanged ומצב טעינה
│   ├── HeroesListViewModel.cs    # לוגיקת מסך האוסף, חיפוש וניווט
│   ├── AddHeroViewModel.cs       # לוגיקת מסך הוספת גיבור ואימות טופס
│   └── HeroDetailsViewModel.cs   # לוגיקת מסך פרטים, עדכון ומחיקה (IQueryAttributable)
├── Views/
│   ├── HeroesListPage.xaml(.cs)  # מסך 1: צפייה באוסף וחיפוש
│   ├── AddHeroPage.xaml(.cs)     # מסך 2: טופס הוספת רשומה חדשה
│   └── HeroDetailsPage.xaml(.cs) # מסך 3: צפייה בפרטי רשומה, עדכון ומחיקה
└── AppShell.xaml(.cs)            # ניהול מעטפת האפליקציה ורישום נתיבי ניווט (Routes)
```

---

3. צעד 1: מודל הנתונים — `Models/HeroItem.cs`

מטרת הקובץ ומקומו בארכיטקטורה

המודל מייצג רשומה בודדת של גיבור-על בארכיון. בחרנו במודל עשיר הכולל טיפוסי נתונים מגוונים (`int`, `string`, `bool`) כדי להכין את הקרקע למיפוי עמודות בטבלת SQLite בהמשך. כל תמונות הגיבורים נטענות מכתובות URL מאומתות ברשת (מתוך מאגר ה-CDN הציבורי `akabab/superhero-api`).

CSHARP

```csharp
namespace LearnSqlite.Models;

public class HeroItem
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string AlterEgo { get; set; } = string.Empty;
    public string Publisher { get; set; } = "Marvel";
    public int PowerLevel { get; set; } = 75;
    public bool IsEliteMember { get; set; }
    public string ImageUrl { get; set; } = string.Empty;

    // מאפייני עזר לקריאה בלבד עבור תצוגה נקייה ב-XAML
    public string PowerBadgeText => $"⚡ עוצמה: {PowerLevel}/100";
    public string EliteStatusText => IsEliteMember ? "⭐ פועל עצמאית 🛡️" : "חבר צוות עלית";
}
```

שם המאפיין	טיפוס ב-C#	תפקיד עתידי ב-SQLite
`Id`	`int`	מפתח ראשי מספור אוטומטי (`[PrimaryKey, AutoIncrement]`)
`AlterEgo` / `Name`	`string`	עמודות טקסט (`TEXT`) לחיפוש וסינון
`Publisher`	`string`	עמודת קטגוריה (`Marvel` / `DC` / `Independent`)
`PowerLevel`	`int`	עמודת מספר שלם (`INTEGER`) בטווח 1–100
`IsEliteMember`	`bool`	עמודה בוליאנית (`0` או `1` ב-SQLite)
`ImageUrl`	`string`	כתובת URL לתמונת הגיבור המוצגת בפקד `Image`

---

4. צעד 2: חוזה ה-CRUD ומימוש ה-Mockup — `Services/`

4.1 ממשק השירות: `Services/IHeroService.cs`

כל פעולות השירות מוגדרות כא-סינכרוניות (`Task`) ומקבילות בדיוק לפעולות מסד הנתונים שנממש בהמשך עם `SQLiteAsyncConnection`:

CSHARP

```
using LearnSqlite.Models;

namespace LearnSqlite.Services;

public interface IHeroService
{
    Task<List<HeroItem>> GetAllHeroesAsync();
    Task<HeroItem?> GetHeroByIdAsync(int id);
    Task<List<HeroItem>> SearchHeroesAsync(string searchText);
    Task<int> AddHeroAsync(HeroItem hero);
    Task<int> UpdateHeroAsync(HeroItem hero);
    Task<int> DeleteHeroAsync(int id);
}
```

> 💡 **מדוע `AddHeroAsync`, `UpdateHeroAsync` ו-`DeleteHeroAsync` מחזירות `Task<int>`?**
>
> בספריית `sqlite-net-pcl`, מתודות ה-CRUD (`InsertAsync`, `UpdateAsync`, `DeleteAsync`) מחזירות `Task<int>` המייצג את **מספר השורות שהושפעו בטבלה** (`Rows Affected`). הגדרת החתימה הזו כבר בממשק ה-Mockup מבטיחה תאימות של 100% במעבר ל-SQLite!

4.2 מימוש הדמה בזיכרון: `Services/MockHeroService.cs`

כיוון שבשלב זה איננו משתמשים ב-Dependency Injection (DI), כל מסך יוצר מופע של `new MockHeroService()`. כדי שכל 3 המסכים יקראו ויעדכנו את **אותו אוסף רשומות**, הרשימה `_heroes` והמונה `_nextId` מוגדרים כ-`static`:

CSHARP

```
public class MockHeroService : IHeroService
{
    private static readonly List<HeroItem> _heroes = new()
    {
        new HeroItem
        {
            Id = 1,
            Name = "באטמן (Batman)",
            AlterEgo = "ברוס ויין",
            Publisher = "DC",
            PowerLevel = 88,
            IsEliteMember = true,
            ImageUrl = "https://raw.githubusercontent.com/akabab/superhero-
api/0.3.0/api/images/md/70-batman.jpg"
        },
        // ... 5 גיבורים נוספים עם כתובות תמונה מאומתות
    };

    private static int _nextId = 7;

    public async Task<int> AddHeroAsync(HeroItem hero)
    {
        await Task.Delay(150);
        var copy = CloneHero(hero);
        copy.Id = _nextId++;
        _heroes.Add(copy);
        hero.Id = copy.Id;
        return 1;
    }
}
```

מדוע אנו מבצעים `CloneHero` בשליפה ובשמירה?

במסד נתונים אמיתי (כמו SQLite), כאשר שולפים רשומה למסך העריכה ומשנים שדה ב-ViewModel, הרשומה בטבלה **אינה** **משתנה** עד שלוחצים על כפתור "עדכן רשומה" (`UpdateHeroAsync`). שכפול האובייקט (`CloneHero`) ב-`MockHeroService` מחקה בדיוק את ההתנהגות של מסד נתונים אמיתי, כך שאם המשתמש משנה טקסט ולוחץ "חזרה ללא שמירה", הנתון המקורי ברשימה לא נפגע!

---

5. צעד 3: מחלקת הבסיס `ViewModels/BaseViewModel.cs`

מחלקת `BaseViewModel` מממשת את `INotifyPropertyChanged` עם מאפיינים מפורשים (`get`/`set` מלאים ללא קיצורים חבויים) ומספקת ניהול אחיד של:

- `IsLoading` ו-`IsNotLoading` (עבור פקד `ActivityIndicator` ותנאי `CanExecute`).
- `Title` (כותרת המסך).

- `StatusMessage`, `HasStatusMessage` ו-`HasError` (להצגת משוב ושגיאות למשתמש).
- מתודה וירטואלית `OnLoadingStateChanged()` המאפשרת לכל ViewModel יורש לרענן אוטומטית את ה-`CanExecute` של הפקודות שלו ברגע שמתחילה או מסתיימת טעינה.

CSHARP

```
public bool IsLoading
{
    get => _isLoading;
    set
    {
        if (_isLoading != value)
        {
            _isLoading = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(IsNotLoading));
            OnLoadingStateChanged();
        }
    }
}
```

---

6. צעד 4: ניווט רב-מסכי באמצעות `AppShell`

האפליקציה כוללת 3 מסכים המנוהלים על ידי `AppShell`:

1. `HeroesListPage` **(מסך הבית הראשי):** מוגדר כ-`ShellContent` בתוך `AppShell.xaml` עם `FlowDirection="RightToLeft"`.
2. `AddHeroPage` **(מסך הוספה):** נרשם כנתיב ניווט (`Routing.RegisterRoute`) ב-`AppShell.xaml.cs`.
3. `HeroDetailsPage` **(מסך פרטי/עדכון רשומה):** נרשם כנתיב ניווט ב-`AppShell.xaml.cs` ומקבל את מזהה הרשומה (`HeroId`) באמצעות `Dictionary<string, object>` והממשק `IQueryAttributable`.

CSHARP

```
public partial class AppShell : Shell
{
    public AppShell()
    {
        InitializeComponent();

        Routing.RegisterRoute(nameof(AddHeroPage), typeof(AddHeroPage));
        Routing.RegisterRoute(nameof(HeroDetailsPage), typeof(HeroDetailsPage));
    }
}
```

7. אינדקס מדריכי המסכים (שיטת הבלוקים המודולרית)

בהמשך ספר זה מופיעים 3 מדריכי פיתוח מפורטים (פרק לכל מסך), הכוללים הדמיות חזותיות, חלוקה לבלוקים ב-XAML וב-ViewModel, ונקודות בקרה לתלמיד:

1. **מדריך מסך 1 (`01-HeroesListPage-Guide.md`):** מסך הצפייה באוסף, חיפוש וניווט (`HeroesListPage` + `HeroesListViewModel`).
2. **מדריך מסך 2 (`02-AddHeroPage-Guide.md`):** מסך הוספת גיבור חדש ואימות קלט (`AddHeroPage` + `AddHeroViewModel`).
3. **מדריך מסך 3 (`03-HeroDetailsPage-Guide.md`):** מסך פרטי גיבור, קבלת `HeroId` בניווט, עדכון ומחיקה (`HeroDetailsPage` + `HeroDetailsViewModel`).
4. **מדריך שלב 4 (`04-SqliteDatabase-Guide.md`):** חיבור מסד הנתונים המקומי SQLite (`SqliteHeroService`) בענף `sqlite`.

---

8. שלב ב' (ענף `sqlite`): מעבר מ-`MockHeroService` למסד נתונים מקומי `SQLite`

לאחר שווידאנו שכל 3 המסכים וזרימת ה-MVVM פועלים כשורה בענף `master`, יצרנו את ענף `sqlite` וביצענו בו 4 צעדים ממוקדים:

8.1 התקנת חבילות ה-NuGet של SQLite ב-`LearnSqlite.csproj`

- `sqlite-net-pcl` **(גרסה `1.9.172`):** ספריית ה-ORM הקלילה המאפשרת מיפוי מחלקות C# לטבלאות SQLite וביצוע פעולות CRUD א-סינכרוניות באמצעות `SQLiteAsyncConnection`.
- `SQLitePCLRaw.bundle_green` **(גרסה `2.1.11`):** חבילת הספקים הטבעיים (Native C/C++ binaries) הנדרשת להרצת מנוע SQLite על גבי Android, Windows, iOS ו-MacCatalyst.

8.2 עיטור המודל `HeroItem` בתכונות מיפוי לטבלה (Attributes)

במחלקה `LearnSqlite/Models/HeroItem.cs` הוספנו את תכונות ה-ORM של `SQLite`:

- `[Table("Heroes")]` — מגדיר את שם הטבלה במסד הנתונים.
- `[PrimaryKey, AutoIncrement]` מעל `Id` — קובע מפתח ראשי עם מספור אוטומטי ב-SQLite.
- `[MaxLength(100), NotNull]` מעל `Name` — מגדיר אילוץ שדה חובה ואורך מקסימלי.
- `[Ignore]` מעל `PowerBadgeText` ו-`EliteStatusText` — מונע יצירת עמודות בטבלה עבור מאפייני עזר מחושבים לתצוגה.

8.3 מימוש `SqliteHeroService` עם אתחול עצל (`InitAsync`) וזריעת נתונים (Seeding)

במחלקה `LearnSqlite/Services/SqliteHeroService.cs` מימשנו את החוזה `IHeroService` מול הקובץ המקומי `Path.Combine(FileSystem.AppDataDirectory, "HeroesArchive.db3")`:

- **בנאי נקי:** בהתאם לכללי ה-Guardrails של .NET MAUI, הבנאי `SqliteHeroService()` אינו מבצע פעולות קובץ או מסד נתונים.

- **אתחול א-סינכרוני עצל (`InitAsync`):** נקרא בתחילת כל פעולת CRUD, מוגן על ידי `SemaphoreSlim`, יוצר את הטבלה (`CreateTableAsync<HeroItem>()`) ובפעם הראשונה שבה הטבלה ריקה (`CountAsync() == 0`) זורע אוטומטית את 6 גיבורי-העל ההתחלתיים באמצעות `InsertAllAsync`.
- **פעולות ה-CRUD:**
  - `GetAllHeroesAsync()` -> `Table<HeroItem>().OrderByDescending(h => h.PowerLevel).ToListAsync()`
  - `GetHeroByIdAsync(int id)` -> `Table<HeroItem>().Where(h => h.Id == id).FirstOrDefaultAsync()`
  - `SearchHeroesAsync(string searchText)` -> סינון א-סינכרוני לפי שם, זהות סודית או יקום קומיקס
  - `AddHeroAsync(HeroItem hero)` -> `InsertAsync(hero)`
  - `UpdateHeroAsync(HeroItem hero)` -> `UpdateAsync(hero)`
  - `DeleteHeroAsync(int id)` -> `DeleteAsync<HeroItem>(id)`

8.4 החלפת השירות ב-Views ללא שינוי ב-ViewModels או ב-XAML

בזכות העובדה שכל שלושת ה-ViewModels תלויים בממשק `IHeroService` בלבד, המעבר מנתוני דמה בזיכרון למסד נתונים SQLite דרש החלפה של שם המחלקה בלבד ב-Code-Behind של שלושת המסכים (`new MockHeroService()` במקום `new SqliteHeroService()`), ללא שינוי של אף שורת קוד ב-ViewModels או ב-XAML!

פרק הרחבה — מדריך פיתוח מפורט: HeroesListPage-01 (שיטת הבלוקים המודולרית)

מטרת הפרק

- **מה הלומד יראה ויוכל לעשות:** צפייה ברשימת גיבורי-העל מתוך השירות בפקד `CollectionView`, סינון גיבורים לפי טקסט חופשי (שם, זהות סודית או יקום), מעבר למסך הוספת גיבור חדש, ומעבר למסך פרטים/עריכה של גיבור נבחר.
- **עובדות הפרויקט הנוכחיות:** המסך ממומש בקבצים `LearnSqlite/Views/HeroesListPage.xaml`, `LearnSqlite/Views/HeroesListPage.xaml.cs` ו-`LearnSqlite/ViewModels/HeroesListViewModel.cs`.
- **מדוע זו אבן הדרך הראשונה:** זהו מסך הבית הראשי ב-`AppShell.xaml` המדגים את פעולת ה-**Read** (`GetAllHeroesAsync` ו-`SearchHeroesAsync`) ב-CRUD.

מפת הדרך

מצב נוכחי	אבן דרך זו	אבן הדרך הבאה	מושג מפתח
מודל ושירות `MockHeroService` מוכנים	הצגת האוסף ב-`CollectionView` וסינון	מסך הוספת גיבור (`AddHeroPage`)	`ObservableCollection<HeroItem>` ו-`RelativeSource`

מפת הבלוקים של המסך

Block: header-banner

🦸 ארכיון גיבורי-העל

סה"כ גיבורים מוצגים בארכיון: 6

+ גיבור חדש

Block: search-filter

חפש לפי שם, זהות סודית או יקום...

חפש

נקה

Block: collection-list



באטמן (Batman)

ברוס ויין

DC ⚡ עוצמה: 88/100

⭐ חבר צוות עלית

פרטים ←



ספיידרמן (Spider-Man)

פיטר פארקר

Marvel ⚡ עוצמה: 85/100

⭐ חבר צוות עלית

פרטים ←



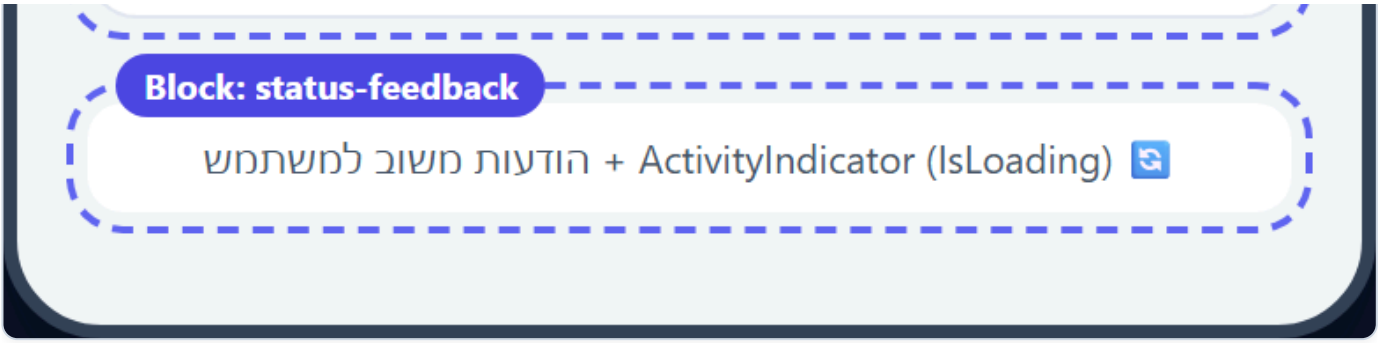
וונדר וומן (Wonder Woman)

דיאנה פרינס

DC ⚡ עוצמה: 95/100

⭐ חבר צוות עלית

פרטים ←



תמונה זו היא מודל הוראה חזותי ב-HTML (אינה צילום אמולטור). המימוש ב-.NET MAUI נבדק ומאומת בנפרד.

בלוק	קובץ ושכבה	מטרה	ראיה / אימות
`header-banner`	`Views/HeroesListPage.xaml`	הצגת כותרת, סיכום כמות (`SummaryText`) וכפתור הוספה	ניווט ל-`AddHeroPage` בלחיצה
`search-filter`	`Views/HeroesListPage.xaml`	שדה קלט לחיפוש וכפתורי "חפש" ו"נקה"	סינון הרשימה דרך `SearchHeroesCommand`
`collection-list`	`Views/HeroesListPage.xaml`	הצגת כרטיסי הגיבורים ב-`CollectionView` עם תמונה מהאינטרנט	הצגת 6 גיבורי הבסיס וכפתור "פרטים"
`status-feedback`	`Views/HeroesListPage.xaml`	חיווי טעינה (`ActivityIndicator`) והודעות שגיאה	מופעל בעת `IsLoading = true`

בלוק 1 — באנר כותרת עליון וניווט להוספה (`header-banner`)

מה משתנה

מוסיפים בחלקו העליון של המסך כרטיס כותרת המציג את סך הגיבורים המוצגים בארכיון וכפתור בולט למעבר למסך הוספת גיבור חדש.

היכן הקוד יושב ומדוע

התצוגה יושבת ב-`LearnSqlite/Views/HeroesListPage.xaml` (שכבת ה-View), בעוד שהטקסט המחושב `SummaryText` ופקודת הניווט `GoToAddHeroCommand` יושבים ב-`LearnSqlite/ViewModels/HeroesListViewModel.cs` (שכבת ה-ViewModel).

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroesListPage.xaml -->
<Border Grid.Row="0"
        StrokeShape="RoundRectangle 16"
        BackgroundColor="#1E3A8A"
        StrokeThickness="0"
        Padding="16">
    <Grid ColumnDefinitions="*,Auto"
          ColumnSpacing="12">
```

```
        <VerticalStackLayout Grid.Column="0"
                             Spacing="4">
            <Label Text="🦸 ארכיון גיבורי-העל"
                   FontSize="20"
                   FontAttributes="Bold"
                   TextColor="White" />
            <Label Text="{Binding SummaryText}"
                   FontSize="13"
                   TextColor="#BFDBFE" />
        </VerticalStackLayout>

        <Button Grid.Column="1"
                Text="+ גיבור חדש"
                Command="{Binding GoToAddHeroCommand}"
                BackgroundColor="#2563EB"
                TextColor="White"
                FontAttributes="Bold"
                CornerRadius="10"
                VerticalOptions="Center" />
    </Grid>
</Border>
```

CSHARP

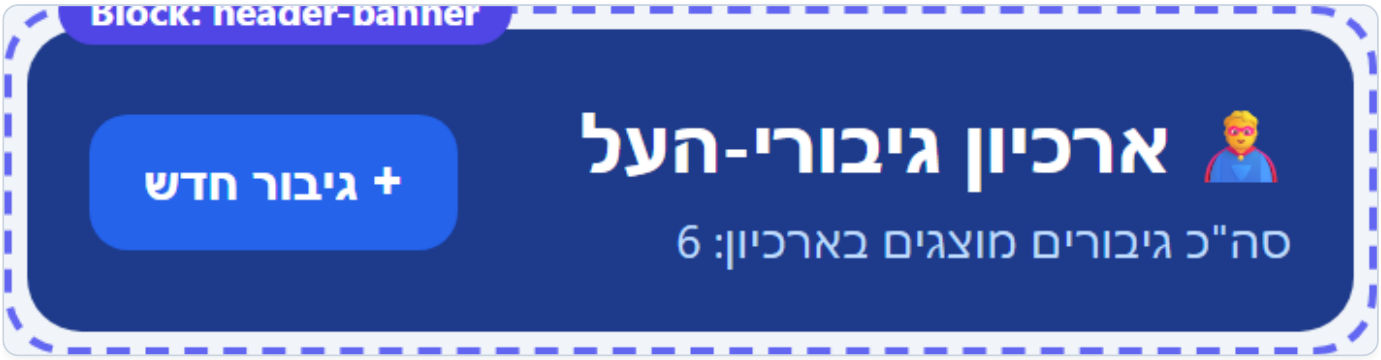
```
// מתוך LearnSqlite/ViewModels/HeroesListViewModel.cs
public int TotalCount
{
    get ⇒ _totalCount;
    set
    {
        if (_totalCount ≠ value)
        {
            _totalCount = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(SummaryText));
        }
    }
}

public string SummaryText ⇒ $"סה\"כ גיבורים מוצגים בארכיון: {TotalCount}";

private async Task GoToAddHeroAsync()
{
    if (Shell.Current is not null)
    {
        await Shell.Current.GoToAsync(nameof(AddHeroPage));
    }
}
```

מדוע זה עובד

כאשר `TotalCount` מתעדכן בסיום טעינת הרשימה, הוא מפעיל `OnPropertyChanged(nameof(SummaryText))` והתווית בבאנר מתעדכנת מיד. כפתור ההוספה מחובר ל-`GoToAddHeroCommand` המוגן על ידי `CanExecute: () ⇒ IsNotLoading`.



נקודת בקרה לתלמיד

> 💡 מדוע קראנו ל-`OnPropertyChanged(nameof(SummaryText))` בתוך ה-`set` של `TotalCount`? מה היה קורה לתווית במסך אם היינו קוראים רק ל-`OnPropertyChanged()` עבור `TotalCount`?

אימות

- בעת עליית המסך מוצג הכיתוב: `סה"כ גיבורים מוצגים בארכיון: 6`.
- לחיצה על `גיבור חדש +` מנווטת למסך `AddHeroPage`.

בלוק 2 — סרגל חיפוש וסינון רשומות (`search-filter`)

מה משתנה

מוסיפים תיבת חיפוש (`Entry`) המקושרת ב-Two-Way Binding למאפיין `SearchText`, לצד כפתור "חפש" וכפתור "נקה".

היכן הקוד יושב ומדוע

הקלט והכפתורים נמצאים ב-`LearnSqlite/Views/HeroesListPage.xaml`, והלוגיקה שקוראת ל-`_heroService.SearchHeroesAsync(SearchText)` נמצאת ב-`LearnSqlite/ViewModels/HeroesListViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

```xml
<!-- מתוך LearnSqlite/Views/HeroesListPage.xaml ⟶ -->
<Border Grid.Row="1"
        StrokeShape="RoundRectangle 12"
        BackgroundColor="White"
        Stroke="#CBD5E1"
```

```
        StrokeThickness="1"
        Padding="10">
    <Grid ColumnDefinitions="*,Auto,Auto"
          ColumnSpacing="8">
        <Entry Grid.Column="0"
               Placeholder="חפש לפי שם, זהות סודית או יקום (Marvel/DC).  "
               Text="{Binding SearchText}"
               TextColor="#0F172A"
               PlaceholderColor="#64748B" />

        <Button Grid.Column="1"
                Text="חפש"
                Command="{Binding SearchHeroesCommand}"
                BackgroundColor="#0284C7"
                TextColor="White"
                CornerRadius="8" />

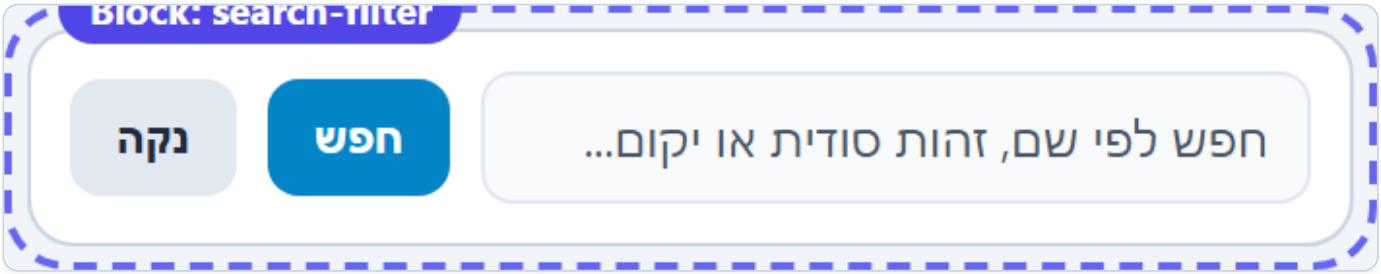
        <Button Grid.Column="2"
                Text="נקה"
                Command="{Binding ClearSearchCommand}"
                BackgroundColor="#E2E8F0"
                TextColor="#1E293B"
                CornerRadius="8" />
    </Grid>
</Border>
```

CSHARP

```
// ... מתוך LearnSqlite/ViewModels/HeroesListViewModel.cs
public async Task ClearSearchAsync()
{
    SearchText = string.Empty;
    await LoadHeroesAsync();
}
```

מדוע זה עובד

פקודת `SearchHeroesCommand` מפעילה את `LoadHeroesAsync()`, שבודקת האם `SearchText` ריק: אם הוא ריק נשלפים כל הגיבורים (`GetAllHeroesAsync`), ואם הוזן טקסט מופעלת מתודת הסינון בשירות (`SearchHeroesAsync`). בענף ה-SQLite מתודה זו תתורגם לשאילתת `WHERE` במסד הנתונים!



נקודת בקרה לתלמיד

> 💡 מדוע עדיף לבצע את הסינון בתוך השירות (`IHeroService.SearchHeroesAsync`) ולא לסנן רק ברשימה המקומית בתוך ה-ViewModel? חישבו מה יקרה כשנעבור למסד נתונים SQLite עם אלפי רשומות.

אימות

- הקלדת `DC` ולחיצה על "חפש" מציגה רק את גיבורי DC ומעדכנת את המונה ל-4.
- לחיצה על "נקה" מאפסת את תיבת הטקסט ומחזירה את כל 6 הגיבורים.

בלוק 3 — רשימת הגיבורים (`CollectionView`) וניווט לפרטים (`collection-list`)

מה משתנה

מציגים את אוסף הגיבורים (`ObservableCollection<HeroItem>`) באמצעות `CollectionView` עם תבנית כרטיס (`DataTemplate`) הכוללת תמונה מהאינטרנט, פרטי הגיבור וכפתור "פרטים ➡" המעביר את הגיבור הנבחר למסך הפרטים.

היכן הקוד יושב ומדוע

תבנית הכרטיס יושבת ב-`LearnSqlite/Views/HeroesListPage.xaml` עם `x:DataType="models:HeroItem"`, ופקודת המעבר `OpenHeroDetailsCommand` יושבת ב-`HeroesListViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroesListPage.xaml ⟶
<CollectionView Grid.Row="2"
                ItemsSource="{Binding Heroes}"
                SelectionMode="None">
    <CollectionView.ItemTemplate>
        <DataTemplate x:DataType="models:HeroItem">
            <Border StrokeShape="RoundRectangle 14"
                    BackgroundColor="White"
                    Stroke="#E2E8F0"
                    StrokeThickness="1"
                    Padding="12"
                    Margin="0,0,0,10">
                <Grid ColumnDefinitions="72,*,Auto"
                      ColumnSpacing="12">
                    <Border Grid.Column="0"
                            StrokeShape="RoundRectangle 12"
                            StrokeThickness="0"
                            HeightRequest="72"
                            WidthRequest="72"
                            BackgroundColor="#E2E8F0">
                        <Image Source="{Binding ImageUrl}"
                               Aspect="AspectFill"
                               HeightRequest="72"
```

```
                            WidthRequest="72" />
                    </Border>

                    <VerticalStackLayout Grid.Column="1"
                                         Spacing="3"
                                         VerticalOptions="Center">
                        <Label Text="{Binding Name}"
                               FontSize="16"
                               FontAttributes="Bold"
                               TextColor="#0F172A" />
                        <Label Text="{Binding AlterEgo}"
                               FontSize="13"
                               TextColor="#475569" />
                    </VerticalStackLayout>

                    <Button Grid.Column="2"
                            Text="פרטים ←"
                            Command="{Binding Source={RelativeSource AncestorType={x:Type
vm:HeroesListViewModel}}, Path=OpenHeroDetailsCommand}"
                            CommandParameter="{Binding .}"
                            BackgroundColor="#EFF6FF"
                            TextColor="#1D4ED8"
                            FontAttributes="Bold"
                            CornerRadius="10"
                            VerticalOptions="Center" />
                </Grid>
            </Border>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

CSHARP

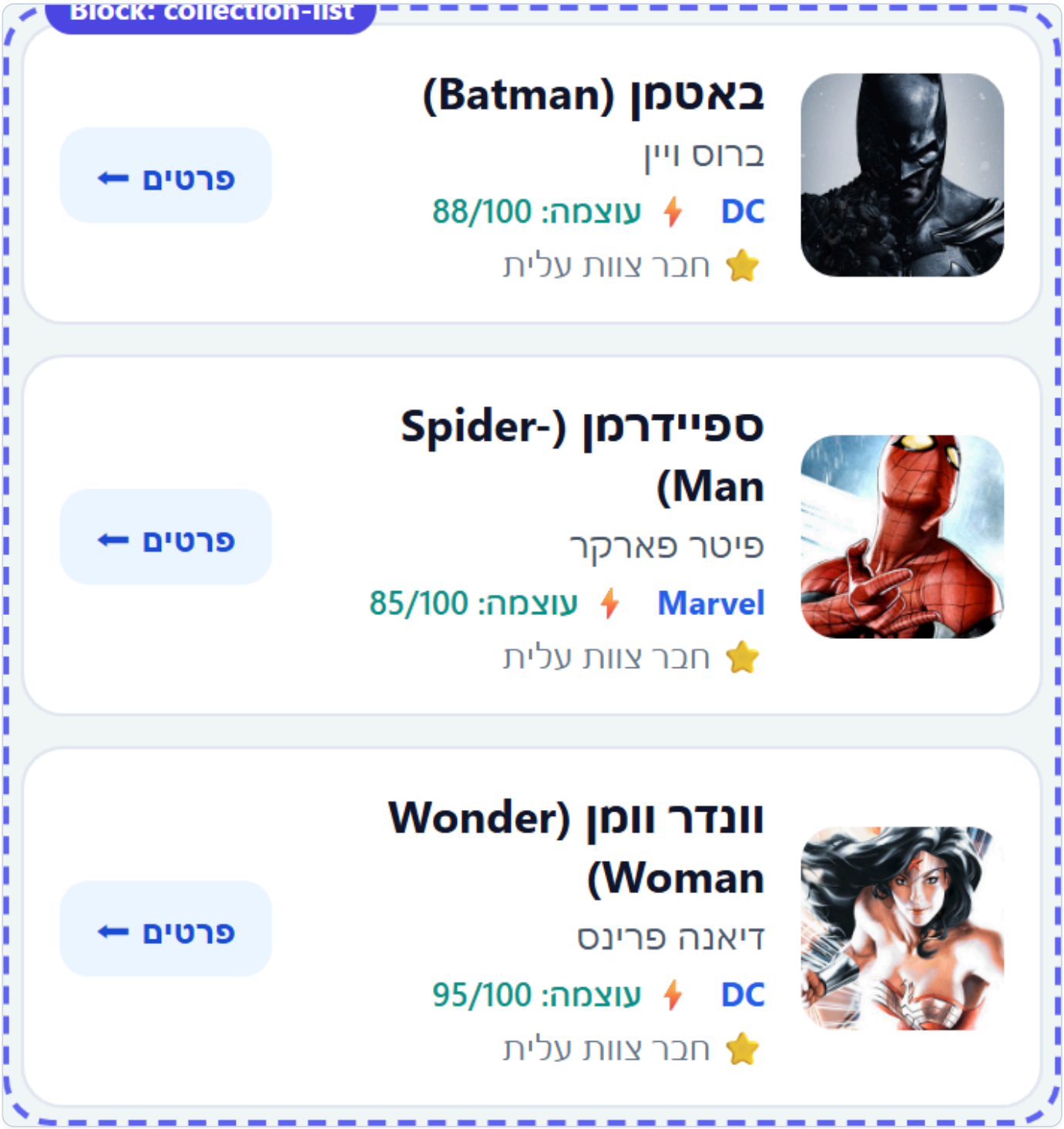
```
// מתוך LearnSqlite/ViewModels/HeroesListViewModel.cs
private async Task OpenHeroDetailsAsync(HeroItem? hero)
{
    if (hero is null || Shell.Current is null)
    {
        return;
    }

    var navigationParams = new Dictionary<string, object>
    {
        { "HeroId", hero.Id }
    };

    await Shell.Current.GoToAsync(nameof(HeroDetailsPage), navigationParams);
}
```

מדוע זה עובד

בתוך `DataTemplate`, הקשר הנתונים (`BindingContext`) הוא אובייקט בודד מסוג `HeroItem`. לכן, כדי שכפתור ה"פרטים" יפעיל פקודה שנמצאת ב-`HeroesListViewModel`, אנו משתמשים ב-`RelativeSource AncestorType={x:Type vm:HeroesListViewModel}` ומעבירים את הפריט הנוכחי דרך `CommandParameter="{Binding .}"`. בנוסף, אנו מעבירים בניווט רק את המזהה `HeroId` (ולא את האובייקט כולו) כדי שמסך הפרטים ישלוף את הרשומה העדכנית מהשירות באמצעות `GetHeroByIdAsync(id)` — הרגל קריטי לעבודה נכונה עם SQLite!



נקודת בקרה לתלמיד

> 💡 מדוע העברנו בניווט למסך הפרטים את המזהה `HeroId` (`int`) במקום להעביר את כל האובייקט `HeroItem`? איזו מתודה ב-`IHeroService` אנחנו מתרגלים בזכות הבחירה הזו?

אימות

- לחיצה על כפתור `פרטים ←` בכרטיס של באטמן מנווטת אל `HeroDetailsPage` ומעבירה `HeroId = 1`.

בלוק 4 — חיווי טעינה (`ActivityIndicator`) וטעינה אוטומטית ב-`OnAppearing` (`status-feedback`)

מה משתנה

מוסיפים בתחתית המסך פקד `ActivityIndicator` ותווית שגיאה/משוב, ומפעילים את `LoadHeroesAsync()` מתוך `OnAppearing()` בקובץ ה-Code-Behind של המסך.

היכן הקוד יושב ומדוע

החיווי הוויזואלי יושב ב-`HeroesListPage.xaml`, וקריאת המחזוריות של המסך יושבת ב-`HeroesListPage.xaml.cs` כיוון ש-`OnAppearing` הוא אירוע מחזור חיים של ה-`ContentPage`.

שינוי הקוד ב-XAML וב-Code-Behind

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroesListPage.xaml -->
<VerticalStackLayout Grid.Row="3"
                     Spacing="6">
    <ActivityIndicator IsRunning="{Binding IsLoading}"
                       IsVisible="{Binding IsLoading}"
                       Color="#2563EB"
                       HorizontalOptions="Center" />

    <Label Text="{Binding StatusMessage}"
           IsVisible="{Binding HasStatusMessage}"
           TextColor="#DC2626"
           FontSize="13"
           HorizontalOptions="Center" />
</VerticalStackLayout>
```

CSHARP

```csharp
// מתוך LearnSqlite/Views/HeroesListPage.xaml.cs
protected override async void OnAppearing()
{
    base.OnAppearing();
```

```
    await _viewModel.LoadHeroesAsync();
}
```

מדוע זה עובד

הפעלת `LoadHeroesAsync()` בתוך `OnAppearing()` (ולא בתוך הבנאי של ה-ViewModel!) מבטיחה שני יתרונות עצומים:

1. איננו חוסמים את יצירת המסך בקריאות א-סינכרוניות בבנאי.
2. בכל פעם שהמשתמש חוזר ממסך ההוספה (`AddHeroPage`) או ממסך העדכון/מחיקה (`HeroDetailsPage`), מתודת `OnAppearing` מופעלת אוטומטית ומרעננת את הרשימה עם הנתונים העדכניים!



נקודת בקרה לתלמיד

💡 מה היה קורה אילו היינו קוראים ל-`LoadHeroesAsync()` רק פעם אחת בבנאי של המסך, ולא ב-`OnAppearing()`, לאחר שהמשתמש הוסיף גיבור חדש במסך ההוספה וחזר אחורה?

אימות

- חזרה ממסך הוספה או ממסך עריכה מרעננת מיד את הרשימה ב-`HeroesListPage`.

רפלקציה והצעד הבא

- **החלטה ארכיטקטונית:** בחרנו להעביר `HeroId` בניווט ולרענן את הרשימה ב-`OnAppearing` כדי שכל מסך יעבוד ישירות מול שירות הנתונים (`IHeroService`), בדיוק כפי שיעבוד מול טבלת SQLite.
- **שאלה למחשבה:** מדוע `Heroes` מוגדר כ-`ObservableCollection<HeroItem>` ולא כ-`List<HeroItem>` רגיל?
- **אתגר קטן:** הוסיפו ב-`HeroesListViewModel` מאפיין מחושב המציג כמה מתוך הגיבורים המוצגים הם חברי צוות עלית (`IsEliteMember == true`).

מקורות

- תיעוד רשמי של Microsoft Learn עבור `.NET MAUI Shell Navigation` ו-`CollectionView`.

פרק הרחבה — מדריך פיתוח מפורט: AddHeroPage-02 (שיטת הבלוקים המודולרית)

מטרת הפרק

- **מה הלומד יראה ויוכל לעשות:** מילוי טופס הוספת גיבור-על חדש עם תצוגה מקדימה חיה של תמונת הגיבור מהאינטרנט, אימות קלט בזמן אמת באמצעות `CanExecute`, ושמירת הרשומה החדשה בשירות הנתונים (`AddHeroAsync`).
- **עובדות הפרויקט הנוכחיות:** המסך ממומש בקבצים `LearnSqlite/Views/AddHeroPage.xaml`, `LearnSqlite/Views/AddHeroPage.xaml.cs` ו-`LearnSqlite/ViewModels/AddHeroViewModel.cs`.
- **מדוע זו אבן הדרך השנייה:** המסך מדגים את פעולת ה-**Create** (`INSERT`) ב-CRUD ואת השילוב של פקדי קלט מגוונים (`Entry`, `Picker`, `Slider`, `Switch`) לקראת שמירה במסד הנתונים.

מפת הדרך

מצב נוכחי	אבן דרך זו	אבן הדרך הבאה	מושג מפתח
מסך אוסף (`HeroesListPage`) פעיל	טופס הוספת רשומה חדשה (`AddHeroAsync`)	מסך פרטים, עדכון ומחיקה (`HeroDetailsPage`)	אימות קלט עם `Command.CanExecute`

מפת הבלוקים של המסך

Block: preview-banner



✨ הוספת גיבור-על חדש

מלא את פרטי הגיבור לשמירה במאגר הנתונים

Block: hero-form

שם גיבור-העל (חובה):

קפטן אמריקה (Captain America)

זהות סודית / שם אמיתי (חובה):

סטיב רוג'רס

יקום / הוצאה לאור:

Marvel ▾

דירוג עוצמה נבחר: 80/100

חבר בצוות עלית (הנוקמים / ליגת הצדק)

Block: image-url-picker

כתובת תמונה מהאינטרנט (Image URL):

https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/md/149-captain-america.jpg



תמונה זו היא מודל הוראה חזותי ב-HTML (אינה צילום אמולטור). המימוש ב-.NET MAUI נבדק ומאומת בנפרד.

בלוק	קובץ ושכבה	מטרה	ראיה / אימות
`preview-banner`	`Views/AddHeroPage.xaml`	באנר כותרת עם תצוגה מקדימה חיה של `ImageUrl`	שינוי ה-URL מעדכן מיד את ה-`Image`
`hero-form`	`Views/AddHeroPage.xaml`	שדות הקלט (`Name`, `AlterEgo`, `Publisher`, `PowerLevel`, `IsEliteMember`)	קישור Two-Way למאפייני `AddHeroViewModel`
`image-url-picker`	`Views/AddHeroPage.xaml`	שדה URL לתמונה + 3 כפתורי דמו מהירים	לחיצה על כפתור דמו ממלאת URL תקין
`save-cancel-actions`	`Views/AddHeroPage.xaml`	כפתורי שמירה (`SaveHeroCommand`) וביטול (`CancelCommand`)	כפתור השמירה פעיל רק כשכל שדות החובה מלאים

בלוק 1 — באנר כותרת ותצוגה מקדימה של התמונה (`preview-banner`)

מה משתנה

מוסיפים בראש המסך כרטיס המציג תמונת תצוגה מקדימה (`Image`) המקושרת ישירות ל-`ImageUrl` שב-ViewModel, כך שהתלמיד רואה מיד שהכתובת שהזין תקינה ונטענת מהאינטרנט.

היכן הקוד יושב ומדוע

התצוגה יושבת ב-`LearnSqlite/Views/AddHeroPage.xaml`, והמאפיין `ImageUrl` יושב ב-`LearnSqlite/ViewModels/AddHeroViewModel.cs`.

שינוי הקוד ב-XAML

```
XML
```

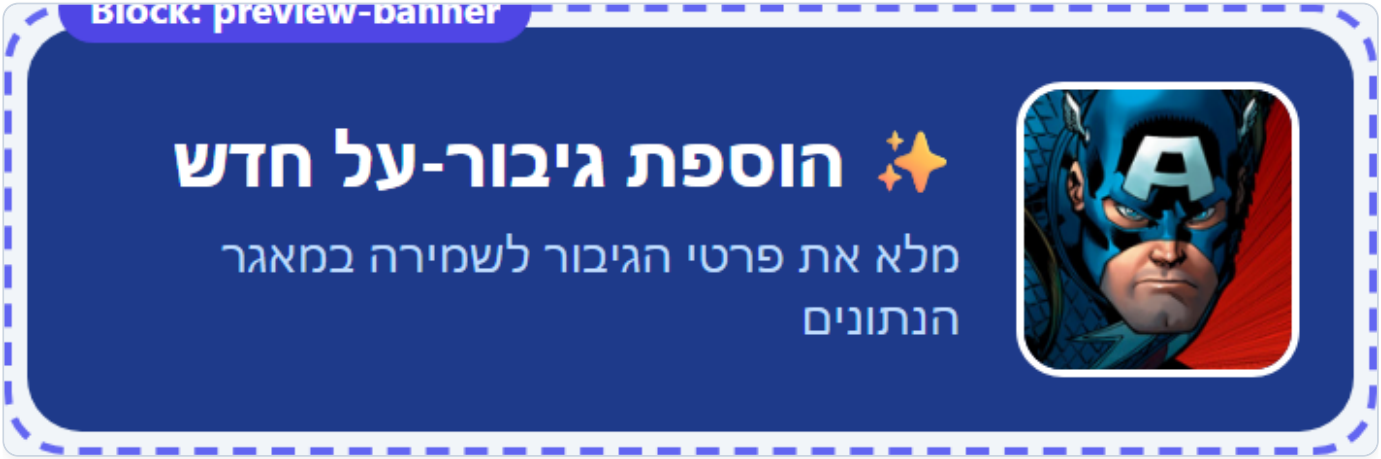
```xaml
<!-- מתוך LearnSqlite/Views/AddHeroPage.xaml -->
<Border StrokeShape="RoundRectangle 16"
        BackgroundColor="#1E3A8A"
        StrokeThickness="0"
        Padding="16">
    <Grid ColumnDefinitions="84,*"
          ColumnSpacing="14">
        <Border Grid.Column="0"
                StrokeShape="RoundRectangle 14"
                Stroke="White"
                StrokeThickness="2"
                HeightRequest="84"
                WidthRequest="84"
                BackgroundColor="#334155">
            <Image Source="{Binding ImageUrl}"
                   Aspect="AspectFill"
                   HeightRequest="84"
                   WidthRequest="84" />
        </Border>

        <VerticalStackLayout Grid.Column="1"
                             Spacing="4"
                             VerticalOptions="Center">
            <Label Text="✨ הוספת גיבור-על חדש"
                   FontSize="19"
                   FontAttributes="Bold"
                   TextColor="White" />
            <Label Text="מלא את פרטי הגיבור לשמירה במאגר הנתונים"
                   FontSize="13"
                   TextColor="#BFDBFE" />
        </VerticalStackLayout>
    </Grid>
</Border>
```

מדוע זה עובד

פקד `Image` ב-.NET MAUI יודע לטעון אוטומטית תמונות מכתובת `https://` כאשר `Source` קשור למחרוזת URL. בכל פעם שערך `ImageUrl` ב-ViewModel משתנה, אירוע `OnPropertyChanged` גורם לפקד ה-`Image` לרענן את התמונה המוצגת.

Block: preview-banner



נקודת בקרה לתלמיד

> 💡 כיצד פקד ה-`Image` בראש המסך יודע להתעדכן מיד כאשר המשתמש מדביק כתובת URL חדשה בתיבת הטקסט שבתחתית הטופס?

אימות

- עם פתיחת המסך מוצגת תמונת ברירת המחדל של קפטן אמריקה מתוך ה-CDN.

בלוק 2 — טופס הזנת פרטי הגיבור (`hero-form`)

מה משתנה

בונים את כרטיס הקלט המרכזי הכולל `Entry` לשם ולזהות הסודית, `Picker` לבחירת יקום הקומיקס, `Slider` לבחירת דירוג עוצמה (1–100), ו-`Switch` לבחירת שייכות לצוות עלית.

היכן הקוד יושב ומדוע

פקדי הקלט נמצאים ב-`LearnSqlite/Views/AddHeroPage.xaml`, ומאפייני ה-State ואימות הקלט (`CanSaveHero`) נמצאים ב-`LearnSqlite/ViewModels/AddHeroViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/AddHeroPage.xaml -->
<Border StrokeShape="RoundRectangle 14"
        BackgroundColor="White"
        Stroke="#CBD5E1"
        StrokeThickness="1"
        Padding="16">
    <VerticalStackLayout Spacing="12">
        <Label Text="שם גיבור-העל (חובה):"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#1E293B" />
        <Entry Placeholder="למשל: קפטן אמריקה (Captain America)"
               Text="{Binding Name}"
               TextColor="#0F172A" />

        <Label Text="זהות סודית / שם אמיתי (חובה):"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#1E293B" />
        <Entry Placeholder="למשל: סטיב רוג'רס"
               Text="{Binding AlterEgo}"
               TextColor="#0F172A" />

        <Label Text="יקום / הוצאה לאור:"
               FontSize="13"
```

```
                FontAttributes="Bold"
                TextColor="#1E293B" />
        <Picker Title="בחר יקום קומיקס"
                ItemsSource="{Binding AvailablePublishers}"
                SelectedItem="{Binding Publisher}"
                TextColor="#0F172A" />

        <Label Text="{Binding PowerLevelDisplay}"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#0D9488" />
        <Slider Minimum="1"
                Maximum="100"
                Value="{Binding PowerLevel}"
                MinimumTrackColor="#0D9488"
                ThumbColor="#0F766E" />
    </VerticalStackLayout>
</Border>
```

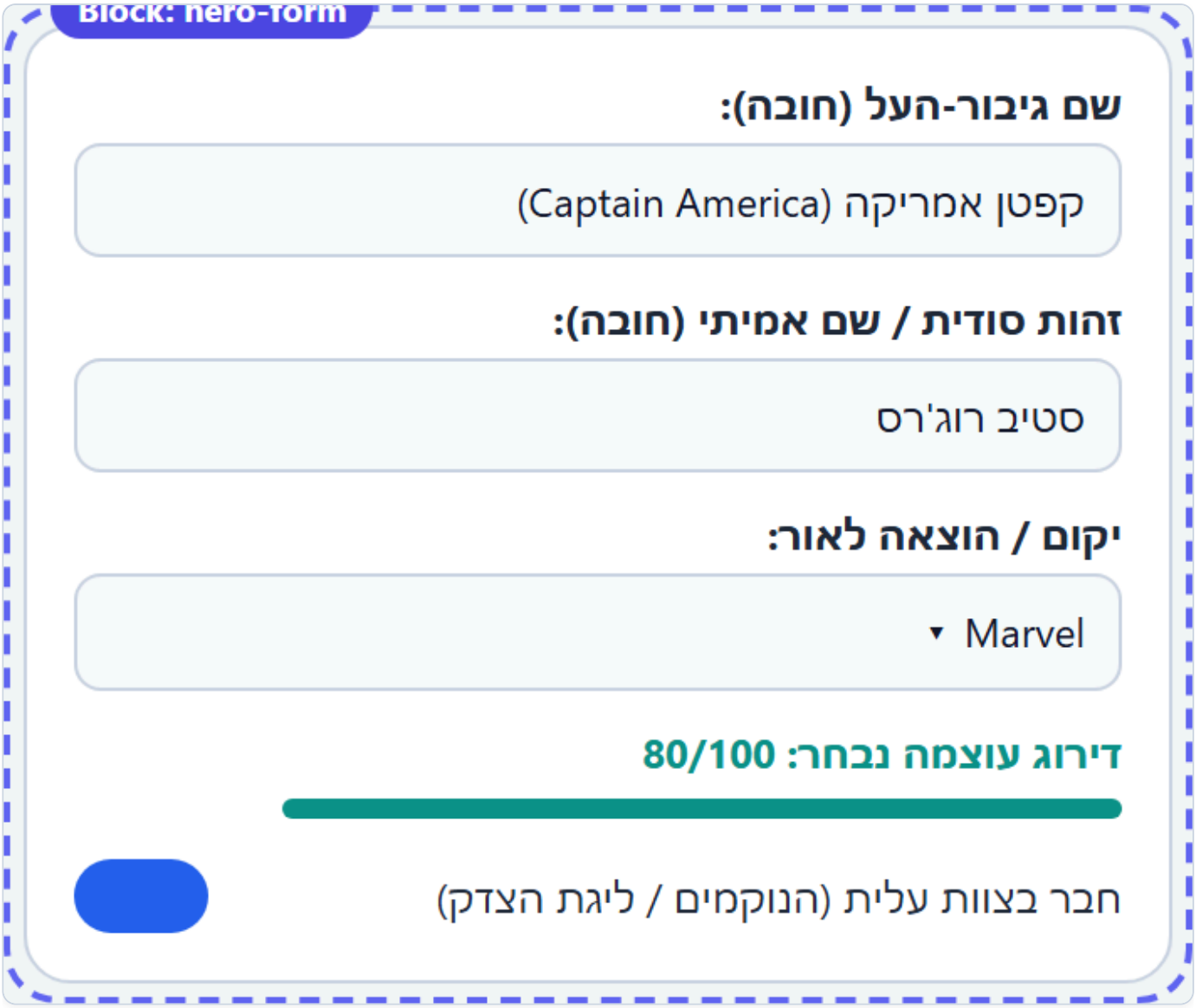
CSHARP

```
// מתוך LearnSqlite/ViewModels/AddHeroViewModel.cs
public string Name
{
    get ⇒ _name;
    set
    {
        if (_name ≠ value)
        {
            _name = value;
            OnPropertyChanged();
            RefreshCommands();
        }
    }
}

public bool CanSaveHero()
{
    return IsNotLoading &&
           !string.IsNullOrWhiteSpace(Name) &&
           !string.IsNullOrWhiteSpace(AlterEgo) &&
           !string.IsNullOrWhiteSpace(ImageUrl) &&
           PowerLevel ⩾ 1 &&
           PowerLevel ⩽ 100;
}
```

מדוע זה עובד

בכל שינוי של `Name` , `AlterEgo` , או `ImageUrl` , מתודת ה- `set` קוראת ל- `RefreshCommands()` , המפעילה את `((Command)SaveHeroCommand).ChangeCanExecute()` . כך כפתור ה"שמור גיבור" נפתח ללחיצה אך ורק כאשר כל שדות החובה הוזנו כראוי — ללא צורך בשימוש ב- `IsEnabled` ידני ב-XAML!



נקודת בקרה לתלמיד

מדוע אנו קוראים ל-`RefreshCommands()` בתוך ה-`set` של `Name` ו-`AlterEgo`? מה היה קורה לכפתור השמירה אם היינו שוכחים שורה זו?

אימות

- כאשר שדה `Name` או `AlterEgo` ריק, כפתור `שמור גיבור` 💾 מושבת (Disabled).
- מילוי שני השדות מפעיל מיד את כפתור השמירה.

בלוק 3 — כתובת תמונה מהאינטרנט וכפתורי בחירה מהירה (`image-url-picker`)

מה משתנה

מוסיפים שדה `Entry` עבור `ImageUrl` (עם `FlowDirection="LeftToRight"` כנדרש עבור כתובות אינטרנט באנגלית) ו-3 כפתורי עזר המאפשרים לתלמיד למלא בלחיצה אחת כתובות תמונה תקינות מתוך מאגר Superhero-API.

היכן הקוד יושב ומדוע

הכפתורים נמצאים ב-`AddHeroPage.xaml` ומעבירים `CommandParameter` אל הפקודה `UsePresetHeroImageCommand` ב-`AddHeroViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/AddHeroPage.xaml -->
<Entry Placeholder="https://..."
       Text="{Binding ImageUrl}"
       FlowDirection="LeftToRight"
       TextColor="#0F172A" />

<HorizontalStackLayout Spacing="8">
    <Button Text="קפטן אמריקה"
            Command="{Binding UsePresetHeroImageCommand}"
            CommandParameter="CaptainAmerica"
            BackgroundColor="#E0F2FE"
            TextColor="#0369A1"
            FontSize="12"
            CornerRadius="8" />
    <Button Text="וולברין"
            Command="{Binding UsePresetHeroImageCommand}"
            CommandParameter="Wolverine"
            BackgroundColor="#FEF3C7"
            TextColor="#B45309"
            FontSize="12"
            CornerRadius="8" />
    <Button Text="ת'ור"
            Command="{Binding UsePresetHeroImageCommand}"
            CommandParameter="Thor"
            BackgroundColor="#EDE9FE"
            TextColor="#6D28D9"
            FontSize="12"
            CornerRadius="8" />
</HorizontalStackLayout>
```

CSHARP

```csharp
// מתוך LearnSqlite/ViewModels/AddHeroViewModel.cs
private void ApplyPresetHeroImage(string? presetKey)
{
    ImageUrl = presetKey switch
    {
        "Wolverine" => "https://raw.githubusercontent.com/akabab/superhero-api/0.3.0/api/images/md/717-wolverine.jpg",
        "Thor" => "https://raw.githubusercontent.com/akabab/superhero-api/0.3.0/api/images/md/659-thor.jpg",
        _ => "https://raw.githubusercontent.com/akabab/superhero-api/0.3.0/api/images/md/149-captain-america.jpg"
```

```
    };
}
```

מדוע זה עובד

שימוש ב-`Command<string>` עם `CommandParameter` מאפשר לנו להשתמש בפקודה אחת בלבד ב-ViewModel עבור מספר כפתורי בחירה מהירה, במקום לשכפל שלוש פקודות נפרדות.



נקודת בקרה לתלמיד

מדוע הגדרנו `FlowDirection="LeftToRight"` על תיבת ה-`Entry` של כתובת ה-URL, למרות שהמסך כולו מוגדר כ-`RightToLeft`?

אימות

- לחיצה על "וולברין" או "ת'ור" מחליפה את ה-URL בתיבה ומעדכנת מיד את התמונה בראש המסך.

בלוק 4 — שמירה בשירות הנתונים וחזרה לארכיון (`save-cancel-actions`)

מה משתנה

מחברים את כפתור השמירה לפקודת `SaveHeroCommand`, היוצרת מופע `HeroItem` חדש, שולחת אותו ל-`_heroService.AddHeroAsync(newHero)`, ומנווטת חזרה למסך האוסף באמצעות `Shell.Current.GoToAsync("..")`.

היכן הקוד יושב ומדוע

הכפתורים יושבים ב-`AddHeroPage.xaml`, ולוגיקת השמירה והניווט חזור יושבת ב-`AddHeroViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/AddHeroPage.xaml ⟶
<Grid ColumnDefinitions="*,*"
      ColumnSpacing="12">
    <Button Grid.Column="0"
            Text="💾 שמור גיבור"
            Command="{Binding SaveHeroCommand}"
            BackgroundColor="#16A34A"
            TextColor="White"
            FontAttributes="Bold"
            CornerRadius="12"
            HeightRequest="48" />

    <Button Grid.Column="1"
            Text="ביטול וחזרה"
            Command="{Binding CancelCommand}"
            BackgroundColor="#E2E8F0"
            TextColor="#1E293B"
            FontAttributes="Bold"
            CornerRadius="12"
            HeightRequest="48" />
</Grid>
```

CSHARP

```csharp
// מתוך LearnSqlite/ViewModels/AddHeroViewModel.cs
public async Task SaveHeroAsync()
{
    if (!CanSaveHero())
    {
        return;
    }

    try
    {
        IsLoading = true;
        var newHero = new HeroItem
        {
            Name = Name.Trim(),
            AlterEgo = AlterEgo.Trim(),
            Publisher = Publisher.Trim(),
            PowerLevel = PowerLevel,
            IsEliteMember = IsEliteMember,
            ImageUrl = ImageUrl.Trim()
        };

        int rowsAffected = await _heroService.AddHeroAsync(newHero);
        if (rowsAffected > 0 && Shell.Current is not null)
        {
            await Shell.Current.GoToAsync("..");
```

```
        }
    }
    finally
    {
        IsLoading = false;
    }
}
```

מדוע זה עובד

הניווט `".."` ב-Shell מסיר את מסך ההוספה ממחסנית הניווט ומחזיר את המשתמש אל `HeroesListPage`. ברגע ש-`HeroesListPage` מופיע מחדש, מתודת `OnAppearing` שלו מופעלת והגיבור החדש מופיע מיד ברשימה!



נקודת בקרה לתלמיד

מדוע איננו קובעים את השדה `Id` של `newHero` בתוך `AddHeroViewModel`? מי אחראי לקבוע את ה-`Id` עכשיו ב-`MockHeroService`, ומי יקבע אותו כשנעבור ל-SQLite?

אימות

- מילוי פרטי גיבור חדש ולחיצה על `שמור גיבור` מחזירה למסך האוסף ומגדילה את מספר הגיבורים מ-6 ל-7.

רפלקציה והצעד הבא

- **החלטה ארכיטקטונית:** השארנו את הקצאת ה-`Id` לשירות הנתונים (ולא ל-ViewModel) משום שב-SQLite המפתח הראשי נקבע אוטומטית ע"י מסד הנתונים (`[PrimaryKey, AutoIncrement]`).
- **שאלה למחשבה:** מדוע עטפנו את הקריאה ל-`_heroService.AddHeroAsync` בבלוק `try-catch-finally` שבו `IsLoading = false` מתבצע תמיד ב-`finally`?
- **אתגר קטן:** הוסיפו תנאי ב-`CanSaveHero()` המוודא שכתובת התמונה מתחילה ב-`https://`.

מקורות

- תיעוד רשמי של Microsoft Learn עבור `.NET MAUI Commanding` ו-`Data Binding`.

פרק הרחבה — מדריך פיתוח מפורט: HeroDetailsPage-03 (שיטת הבלוקים המודולרית)

מטרת הפרק

- **מה הלומד יראה ויוכל לעשות:** קבלת מזהה רשומה (`HeroId`) מניווט ה-Shell באמצעות `IQueryAttributable`, שליפת הרשומה המלאה מהשירות (`GetHeroByIdAsync`), עריכת השדות ושמירתם (`UpdateHeroAsync`), או מחיקת הרשומה מהמאגר (`DeleteHeroAsync`).
- **עובדות הפרויקט הנוכחיות:** המסך ממומש בקבצים `LearnSqlite/Views/HeroDetailsPage.xaml`, `LearnSqlite/Views/HeroDetailsPage.xaml.cs` ו-`LearnSqlite/ViewModels/HeroDetailsViewModel.cs`.
- **מדוע זו אבן הדרך השלישית:** המסך משלים את מחזור ה-CRUD המלא (**Read by ID**, **Update**, **Delete**) ומדגים כיצד מפתח ראשי (`Id`) משמש לאיתור, עדכון ומחיקת שורה ספציפית בטבלה.

מפת הדרך

מצב נוכחי	אבן דרך זו	אבן הדרך הבאה	מושג מפתח
צפייה באוסף והוספת רשומה פועלות	שליפה לפי `HeroId`, עדכון (`Update`) ומחיקה (`Delete`)	מעבר לענף ה-SQLite (`SqliteHeroService`)	`IQueryAttributable` ועבודה מול מפתח ראשי (`Primary Key`)

מפת הבלוקים של המסך

Block: identity-header



באטמן (Batman)

ברוס ויין

מזהה רשומה (Primary Key): #1

Block: edit-fields

שם גיבור-העל:

באטמן (Batman)

זהות סודית / שם אמיתי:

ברוס ויין

יקום / הוצאה לאור:

DC ▾

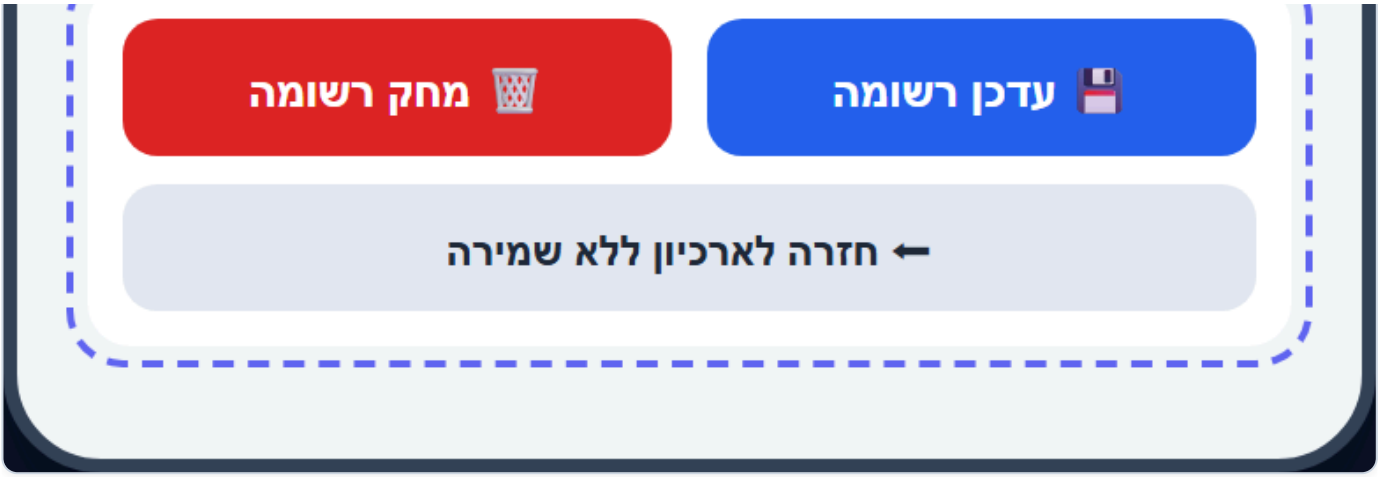
דירוג עוצמה: 88/100

חבר בצוות עלית (הנוקמים / ליגת הצדק)

כתובת תמונה מהאינטרנט (Image URL):

https://cdn.jsdelivr.net/gh/akabab/superhero-api@0.3.0/api/images/md/70-batman.jpg

Block: update-delete-actions



תמונה זו היא מודל הוראה חזותי ב-HTML (אינה צילום אמולטור). המימוש ב-.NET MAUI נבדק ומאומת בנפרד.

בלוק	קובץ ושכבה	מטרה	ראיה / אימות
`identity-header`	`Views/HeroDetailsPage.xaml`	הצגת תמונת הגיבור, שמו ומזהה הרשומה (`#Id`)	טעינת הגיבור לפי `HeroId` בכניסה למסך
`edit-fields`	`Views/HeroDetailsPage.xaml`	טופס עריכת שדות הרשומה הקיימת	שינוי ערכים מעדכן את המאפיינים ב-ViewModel
`update-delete-actions`	`Views/HeroDetailsPage.xaml`	כפתורי "עדכן רשומה", "מחק רשומה" ו"חזרה ללא שמירה"	עדכון או מחיקה משתקפים מיד בחזרה לארכיון

בלוק 1 — קבלת `HeroId` בניווט וכרטיס זהות הרשומה (`identity-header`)

מה משתנה

מממשים ב-`HeroDetailsViewModel` את הממשק `IQueryAttributable` כדי לקבל את הפרמטר `HeroId` שנשלח ממסך האוסף, שולפים את הגיבור באמצעות `_heroService.GetHeroByIdAsync(id)`, ומציגים בראש המסך את תמונתו ואת מזהה המפתח הראשי שלו (`RecordIdDisplay`).

היכן הקוד יושב ומדוע

הכרטיס העליון יושב ב-`LearnSqlite/Views/HeroDetailsPage.xaml`, וקליטת פרמטר הניווט יושבת ב-`LearnSqlite/ViewModels/HeroDetailsViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroDetailsPage.xaml -->
<Border StrokeShape="RoundRectangle 16"
        BackgroundColor="#0F172A"
        StrokeThickness="0"
        Padding="16">
    <Grid ColumnDefinitions="96,*"
          ColumnSpacing="14">
```

```
        <Border Grid.Column="0"
                StrokeShape="RoundRectangle 14"
                Stroke="#38BDF8"
                StrokeThickness="2"
                HeightRequest="96"
                WidthRequest="96"
                BackgroundColor="#1E293B">
            <Image Source="{Binding ImageUrl}"
                   Aspect="AspectFill"
                   HeightRequest="96"
                   WidthRequest="96" />
        </Border>

        <VerticalStackLayout Grid.Column="1"
                             Spacing="4"
                             VerticalOptions="Center">
            <Label Text="{Binding Name}"
                   FontSize="19"
                   FontAttributes="Bold"
                   TextColor="White" />
            <Label Text="{Binding AlterEgo}"
                   FontSize="14"
                   TextColor="#CBD5E1" />
            <Label Text="{Binding RecordIdDisplay}"
                   FontSize="12"
                   FontAttributes="Bold"
                   TextColor="#38BDF8" />
        </VerticalStackLayout>
    </Grid>
</Border>
```

CSHARP

```
// המשך LearnSqlite/ViewModels/HeroDetailsViewModel.cs
public async void ApplyQueryAttributes(IDictionary<string, object> query)
{
    if (query.TryGetValue("HeroId", out object? rawId))
    {
        if (rawId is int intId)
        {
            await LoadHeroByIdAsync(intId);
        }
        else if (int.TryParse(rawId?.ToString(), out int parsedId))
        {
            await LoadHeroByIdAsync(parsedId);
        }
    }
}
```

מדוע זה עובד

כאשר מנווטים עם `Shell.Current.GoToAsync(nameof(HeroDetailsPage), navigationParams)`, תשתית ה-Shell בודקת האם ה-`BindingContext` של הדף מממש את `IQueryAttributable`. אם כן, היא מזריקה אליו אוטומטית את מילון הפרמטרים, וכך `LoadHeroByIdAsync` טוען את הרשומה המתאימה מהשירות.



נקודת בקרה לתלמיד

מדוע אנו מציגים את `HeroId` כתווית לקריאה בלבד (`Label`) בראש המסך ולא כשדה `Entry` שהמשתמש יכול לערוך?
מה תפקידו של המפתח הראשי (`Primary Key`) בטבלת SQLite?

אימות

- לחיצה על "פרטים" עבור באטמן מציגה בראש המסך: `מזהה רשומה (Primary Key): #1`.

בלוק 2 — טופס עריכת שדות הרשומה (`edit-fields`)

מה משתנה

מציגים את כל שדות הגיבור שנשלפו (`Name`, `AlterEgo`, `Publisher`, `PowerLevel`, `IsEliteMember`, `ImageUrl`) בתוך פקדי עריכה הדו-כיווניים (`Two-Way Binding`).

היכן הקוד יושב ומדוע

הפקדים יושבים ב-`LearnSqlite/Views/HeroDetailsPage.xaml`, והמאפיינים יושבים ב-`LearnSqlite/ViewModels/HeroDetailsViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroDetailsPage.xaml -->
<Border StrokeShape="RoundRectangle 14"
```

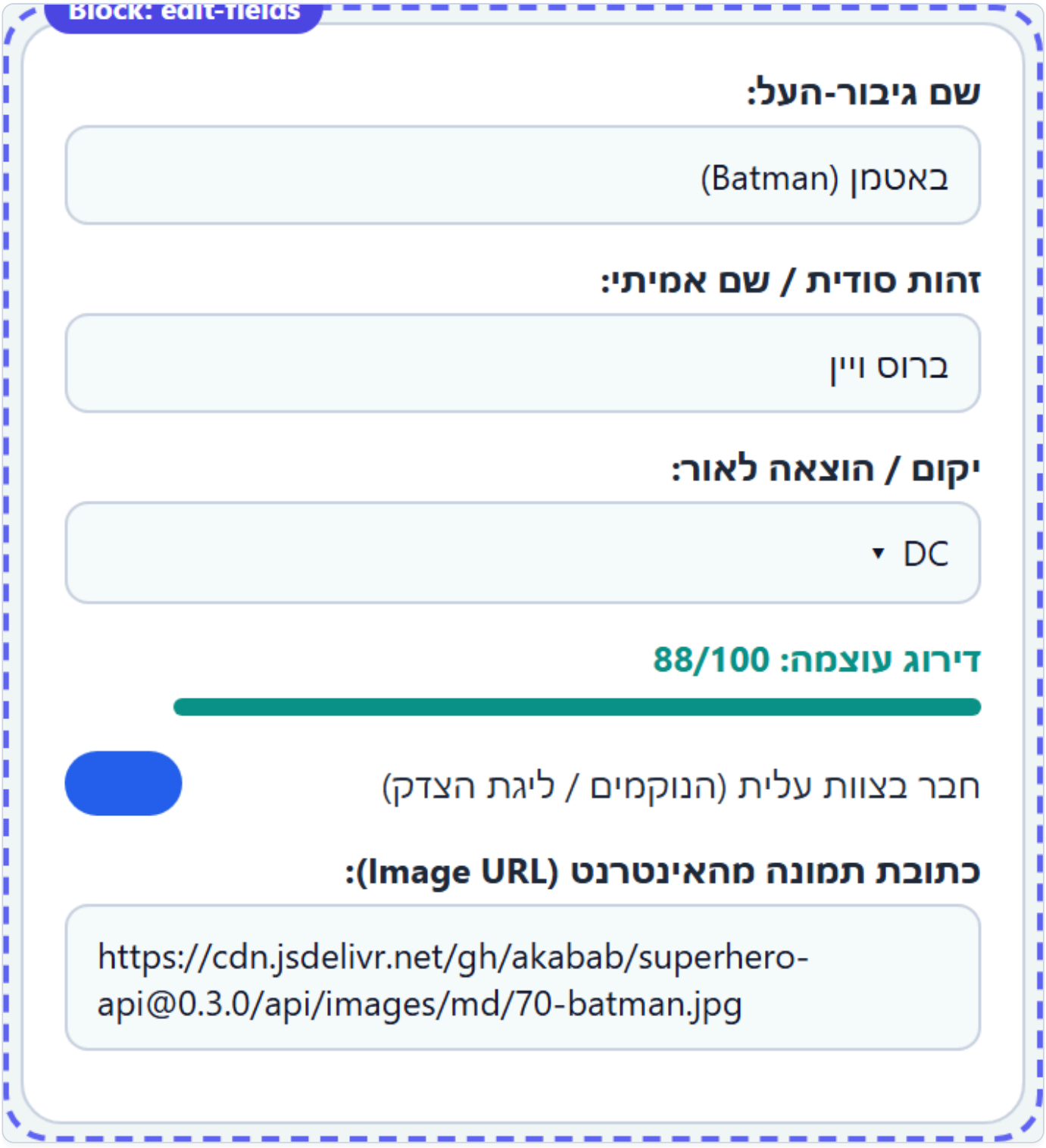
```
        BackgroundColor="White"
        Stroke="#CBD5E1"
        StrokeThickness="1"
        Padding="16">
    <VerticalStackLayout Spacing="12">
        <Label Text="שם גיבור-העל:"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#1E293B" />
        <Entry Text="{Binding Name}"
               Placeholder="שם הגיבור"
               TextColor="#0F172A" />

        <Label Text="זהות סודית / שם אמיתי:"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#1E293B" />
        <Entry Text="{Binding AlterEgo}"
               Placeholder="זהות סודית"
               TextColor="#0F172A" />

        <Label Text="{Binding PowerLevelDisplay}"
               FontSize="13"
               FontAttributes="Bold"
               TextColor="#0D9488" />
        <Slider Minimum="1"
                Maximum="100"
                Value="{Binding PowerLevel}"
                MinimumTrackColor="#0D9488"
                ThumbColor="#0F766E" />
    </VerticalStackLayout>
</Border>
```

מדוע זה עובד

כאשר `LoadHeroByIdAsync` מעתיק את ערכי ה-`HeroItem` שנשלף אל מאפייני ה-ViewModel, כל הפקדים בטופס מתמלאים אוטומטית בערכים הנוכחיים של הרשומה. שינוי של כל שדה על ידי המשתמש מעדכן את ה-ViewModel בזיכרון, אך **עדיין לא משנה את המאגר** עד ללחיצה על "עדכן רשומה".



נקודת בקרה לתלמיד

אם המשתמש משנה את שם הגיבור בטופס ואז לוחץ על `חזרה לארכיון ללא שמירה ←`, מדוע השם המקורי בארכיון נשאר ללא שינוי?

אימות

- כל שדות הטופס מתמלאים בנתוני הגיבור שנבחר מיד עם פתיחת המסך.

בלוק 3 — פעולות עדכון (`Update`) ומחיקה (`Delete`) מול השירות (`update-delete-actions`)

מה משתנה

מוסיפים שני כפתורי פעולה מרכזיים: כפתור **עדכן רשומה** (`UpdateHeroCommand`) הקורא ל-`_heroService.UpdateHeroAsync(updated)`, וכפתור **מחק רשומה** (`DeleteHeroCommand`) הקורא ל-`_heroService.DeleteHeroAsync(HeroId)`.

היכן הקוד יושב ומדוע

הכפתורים יושבים ב-`LearnSqlite/Views/HeroDetailsPage.xaml`, ומתודות ה-Update וה-Delete יושבות ב-`LearnSqlite/ViewModels/HeroDetailsViewModel.cs`.

שינוי הקוד ב-XAML וב-ViewModel

XML

```xml
<!-- מתוך LearnSqlite/Views/HeroDetailsPage.xaml -->
<Grid ColumnDefinitions="*,*"
      ColumnSpacing="12">
    <Button Grid.Column="0"
            Text="💾 עדכן רשומה"
            Command="{Binding UpdateHeroCommand}"
            BackgroundColor="#2563EB"
            TextColor="White"
            FontAttributes="Bold"
            CornerRadius="12"
            HeightRequest="48" />

    <Button Grid.Column="1"
            Text="🗑 מחק רשומה"
            Command="{Binding DeleteHeroCommand}"
            BackgroundColor="#DC2626"
            TextColor="White"
            FontAttributes="Bold"
            CornerRadius="12"
            HeightRequest="48" />
</Grid>
```

CSHARP

```csharp
// מתוך LearnSqlite/ViewModels/HeroDetailsViewModel.cs
public async Task UpdateHeroAsync()
{
    if (!CanUpdateHero())
    {
        return;
    }

    try
```

```
    {
        IsLoading = true;
        var updated = new HeroItem
        {
            Id = HeroId,
            Name = Name.Trim(),
            AlterEgo = AlterEgo.Trim(),
            Publisher = Publisher.Trim(),
            PowerLevel = PowerLevel,
            IsEliteMember = IsEliteMember,
            ImageUrl = ImageUrl.Trim()
        };

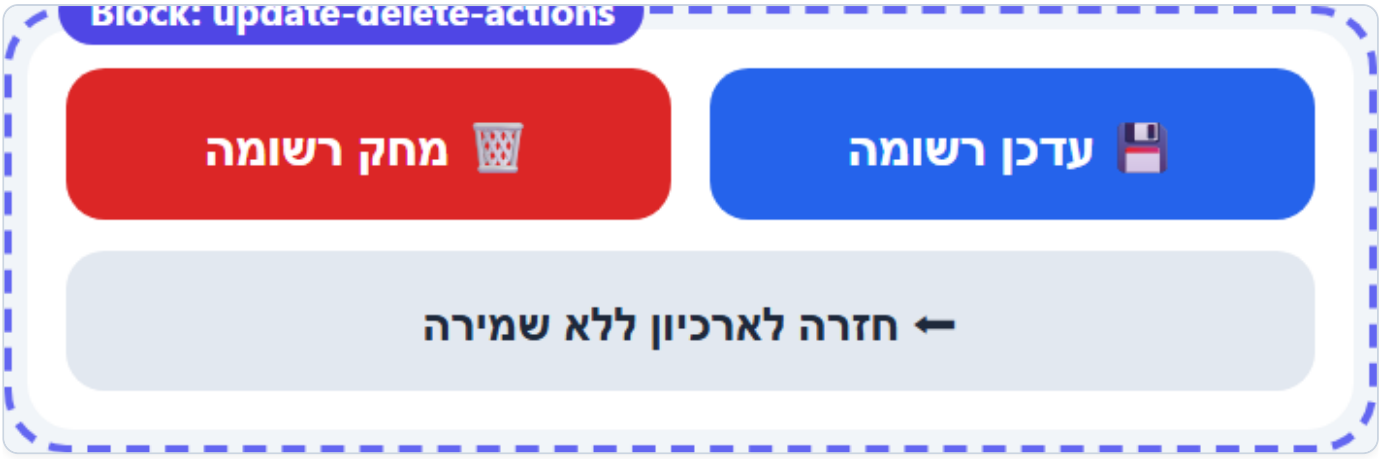
        int rowsAffected = await _heroService.UpdateHeroAsync(updated);
        if (rowsAffected > 0 && Shell.Current is not null)
        {
            await Shell.Current.GoToAsync("..");
        }
    }
    finally
    {
        IsLoading = false;
    }
}

public async Task DeleteHeroAsync()
{
    if (!CanDeleteHero())
    {
        return;
    }

    try
    {
        IsLoading = true;
        int rowsAffected = await _heroService.DeleteHeroAsync(HeroId);
        if (rowsAffected > 0 && Shell.Current is not null)
        {
            await Shell.Current.GoToAsync("..");
        }
    }
    finally
    {
        IsLoading = false;
    }
}
```

מדוע זה עובד

- ב-`UpdateHeroAsync`, אנו בונים אובייקט `HeroItem` הכולל את `Id = HeroId`. זהו פרט קריטי: ב-SQLite, פקודת `UpdateAsync(hero)` מאתרת את השורה לעדכון בטבלה אך ורק לפי ערך ה-`[PrimaryKey]` שלה (`Id`)!
- ב-`DeleteHeroAsync`, אנו מעבירים את `HeroId` אל `_heroService.DeleteHeroAsync(HeroId)` כדי למחוק את הרשומה המתאימה ולחזור למסך האוסף.



נקודת בקרה לתלמיד

💡 מה היה קורה ב-`UpdateHeroAsync()` אילו היינו שוכחים להציב `Id = HeroId` בתוך האובייקט `updated` שאנו שולחים אל `_heroService.UpdateHeroAsync(updated)`?

אימות

- שינוי `PowerLevel` של באטמן ל-99 ולחיצה על `עדכן רשומה` מחזירה למסך האוסף ומציגה `עוצמה: 99/100`.
- לחיצה על `מחק רשומה` מוחקת את הגיבור מהארכיון ומקטינה את המונה ב-1.

רפלקציה והצעד הבא

- **החלטה ארכיטקטונית:** ריכזנו את שתי הפעולות התלויות במפתח ראשי קיים (`Update` ו-`Delete`) במסך הפרטים (`HeroDetailsPage`), כך שה הפרדה בין יצירת רשומה חדשה (`AddHeroPage` ללא `Id`) לבין טיפול ברשומה קיימת (`HeroDetailsPage` עם `HeroId`) ברורה לחלוטין לתלמיד.
- **שאלה למחשבה:** אילו שינויים נצטרך לבצע ב-`HeroesListViewModel`, `AddHeroViewModel` ו-`HeroDetailsViewModel` כאשר נחליף את `MockHeroService` ב-`SqliteHeroService`? (רמז: בדקו האם ה-ViewModels תלויים ב-`MockHeroService` או בממשק `IHeroService`!).
- **אתגר קטן:** הוסיפו כפתור "איפוס שינויים" במסך `HeroDetailsPage` הקורא מחדש ל-`LoadHeroByIdAsync(HeroId)` ומחזיר את השדות לערכם השמור במאגר.

מקורות

- תיעוד רשמי של Microsoft Learn עבור `IQueryAttributable` ב-.NET MAUI Shell.

פרק הרחבה — מדריך פיתוח מפורט: SqliteDatabase-04 (שיטת הבלוקים המודולרית)

1. מטרת השלב והארכיטקטורה

בשלושת השלבים הראשונים (ענף `master`) בנינו את האפליקציה בשלמותה מול שירות דמה בזיכרון (`MockHeroService`) המממש את הממשק `IHeroService`.
בשלב זה (ענף `sqlite`) אנו מחליפים את שכבת הנתונים במסד נתונים מקומי אמיתי מסוג **SQLite** השומר את הנתונים בקובץ קבוע (`HeroesArchive.db3`) על גבי המכשיר — כך שכל הוספה, עריכה או מחיקה נשמרות גם לאחר סגירת האפליקציה והפעלתה מחדש.

מודל ויזואלי של ארכיטקטורת השלב (הדמיית HTML לימודית של 4 הבלוקים במעבר ל-SQLite):

Block: nuget-packages

שלב 1: חבילות ה-NuGet של SQLite ב-.NET MAUI

שילוב ה-ORM הקליל יחד עם ספקי ה-C/C++ הטבעיים לכל הפלטפורמות

`sqlite-net-pcl (1.9.172)`

`SQLitePCLRaw.bundle_green (2.1.11)`

Block: table-model

שלב 2: מיפוי HeroItem לטבלת SQLite באמצעות Attributes

```
[Table("Heroes")]
public class HeroItem {
  [PrimaryKey, AutoIncrement]
  public int Id { get; set; }
  [MaxLength(100), NotNull]
  public string Name { get; set; }
  [Ignore] public string PowerBadgeText => ...
}
```

Block: lazy-init-crud

שלב 3: SqliteHeroService — אתחול עצל (InitAsync) ופעולות CRUD

```
FileSystem.AppDataDirectory + "HeroesArchive.db3"
```

InitAsync() + Seed
יצירת הטבלה CreateTableAsync והזרעת 6 גיבורים אם הטבלה ריקה

Table<HeroItem>()
שליפה וחיפוש א-סינכרוניים עם Where-ו OrderByDescending

InsertAsync(hero)
הוספת רשומה חדשה וקבלת Id אוטומטי מ-SQLite

UpdateAsync / DeleteAsync
עדכון או מחיקה של רשומה קיימת לפי ה-PrimaryKey

Block: service-swap

שלב 4: החלפת השירות דרך הממשק IHeroService (אפס שינויים ב-ViewModel!)

~~`new MockHeroService()`~~ `===` `IHeroService` `===>` `new SqliteHeroService()`

2. פירוק לבלוקים (Step-by-Step Blocks)

בלוק 1: התקנת חבילות ה-NuGet עבור SQLite ב-.NET MAUI



- **קובץ ושכבה:** `LearnSqlite/LearnSqlite.csproj` (הגדרות הפרויקט).
- **מה הקוד עושה:** מוסיף שתי חבילות NuGet משלימות:

1. `sqlite-net-pcl` — ספריית ORM קלילה המאפשרת למפות מחלקות C# לטבלאות SQLite ולבצע פעולות CRUD א-סינכרוניות (`SQLiteAsyncConnection`) ללא כתיבת משפטי SQL ידניים.
2. `SQLitePCLRaw.bundle_green` — חבילת ה-"Batteries Included" המספקת את ספריות ה-C/C++ הטבעיות (`e_sqlite3`) עבור Android, Windows, iOS ו-MacCatalyst.

```xml
<ItemGroup>
    <PackageReference Include="Microsoft.Maui.Controls" Version="$(MauiVersion)" />
    <PackageReference Include="Microsoft.Extensions.Logging.Debug" Version="10.0.0" />
    <PackageReference Include="sqlite-net-pcl" Version="1.9.172" />
    <PackageReference Include="SQLitePCLRaw.bundle_green" Version="2.1.11" />
</ItemGroup>
```

- **נקודת בדיקה לתלמיד (Checkpoint):** מדוע לא מספיק להתקין רק את `sqlite-net-pcl` ב-.NET MAUI, ומה תפקידה של `SQLitePCLRaw.bundle_green` בזמן ריצה על אנדרואיד או Windows?

בלוק 2: עיטור המודל `HeroItem` בתכונות מיפוי לטבלה (SQLite Attributes)



- **קובץ ושכבה:** `LearnSqlite/Models/HeroItem.cs` (שכבת ה-Model).
- **מה הקוד עושה:** מוסיף תכונות (Attributes) מתוך מרחב השמות `SQLite` מעל המחלקה והמאפיינים שלה:
  - `[Table("Heroes")]` — קובע את שם הטבלה במסד הנתונים.

- `[PrimaryKey, AutoIncrement]` מעל `Id` — מגדיר את `Id` כמפתח ראשי שמקבל מספור אוטומטי מ-SQLite בכל `InsertAsync`.
- `[MaxLength(100), NotNull]` מעל `Name` — מגדיר אילוץ אורך וחובה ברמת הטבלה.
- `[Ignore]` מעל `PowerBadgeText` ו-`EliteStatusText` — מורה ל-SQLite לא ליצור עמודות עבור מאפייני תצוגה מחושבים.

CSHARP

```
using SQLite;

namespace LearnSqlite.Models;

[Table("Heroes")]
public class HeroItem
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [MaxLength(100), NotNull]
    public string Name { get; set; } = string.Empty;

    [MaxLength(100)]
    public string AlterEgo { get; set; } = string.Empty;

    [MaxLength(50)]
    public string Publisher { get; set; } = "Marvel";

    public int PowerLevel { get; set; } = 75;

    public bool IsEliteMember { get; set; }

    public string ImageUrl { get; set; } = string.Empty;

    [Ignore]
    public string PowerBadgeText => $"⚡ עוצמה: {PowerLevel}/100";

    [Ignore]
    public string EliteStatusText => IsEliteMember ? "⭐ פועל עצמאית 🛡️" : "חבר צוות עלית";
}
```

- **נקודת בדיקה לתלמיד (Checkpoint):** מה יקרה בעת קריאה ל-`InsertAsync(newHero)` אם נשכח להוסיף `[AutoIncrement]` מעל המאפיין `Id`?

בלוק 3: מימוש `SqliteHeroService` — אתחול עצל (`InitAsync`), זריעת נתונים ופעולות CRUD



- **קובץ ושכבה:** `LearnSqlite/Services/SqliteHeroService.cs` (שכבת ה-Service).
- **מה הקוד עושה:**
  1. מחשב את נתיב קובץ מסד הנתונים באמצעות `Path.Combine(FileSystem.AppDataDirectory, "HeroesArchive.db3")`.
  2. שומר על כלל ה-Guardrail החשוב: **הבנאי (`SqliteHeroService()`) ריק ואינו מבצע פעולות קובץ או מסד נתונים.**
  3. מפעיל מתודת אתחול א-סינכרונית עצלה `InitAsync()` בתחילת כל פעולת CRUD, המוגנת באמצעות `SemaphoreSlim` מפני מצבי מרוץ (Race Conditions), יוצרת את הטבלה (`CreateTableAsync<HeroItem>()`) וזורעת 6 גיבורי-על התחלתיים אם הטבלה ריקה (`CountAsync() == 0`).
  4. מממש את כל 6 המתודות של `IHeroService` מול `SQLiteAsyncConnection`.

CSHARP

```
public class SqliteHeroService : IHeroService
{
    public const string DatabaseFilename = "HeroesArchive.db3";

    public const SQLiteOpenFlags Flags =
        SQLiteOpenFlags.ReadWrite |
        SQLiteOpenFlags.Create |
```

```
        SQLiteOpenFlags.SharedCache;

    public static string DatabasePath =>
        Path.Combine(FileSystem.AppDataDirectory, DatabaseFilename);

    private static SQLiteAsyncConnection? _database;
    private static readonly SemaphoreSlim _initLock = new(1, 1);

    private async Task InitAsync()
    {
        if (_database is not null)
            return;

        await _initLock.WaitAsync();
        try
        {
            if (_database is not null)
                return;

            _database = new SQLiteAsyncConnection(DatabasePath, Flags);
            await _database.CreateTableAsync<HeroItem>();

            int existingCount = await _database.Table<HeroItem>().CountAsync();
            if (existingCount == 0)
            {
                await _database.InsertAllAsync(GetSeedHeroes());
            }
        }
        finally
        {
            _initLock.Release();
        }
    }

    public async Task<List<HeroItem>> GetAllHeroesAsync()
    {
        await InitAsync();
        return await _database!.Table<HeroItem>()
            .OrderByDescending(h => h.PowerLevel)
            .ToListAsync();
    }

    public async Task<HeroItem?> GetHeroByIdAsync(int id)
    {
        await InitAsync();
        return await _database!.Table<HeroItem>()
            .Where(h => h.Id == id)
            .FirstOrDefaultAsync();
    }

    public async Task<int> AddHeroAsync(HeroItem hero)
    {
        await InitAsync();
        return await _database!.InsertAsync(hero);
    }

    public async Task<int> UpdateHeroAsync(HeroItem hero)
    {
        await InitAsync();
```

```
        return await _database!.UpdateAsync(hero);
    }

    public async Task<int> DeleteHeroAsync(int id)
    {
        await InitAsync();
        return await _database!.DeleteAsync<HeroItem>(id);
    }
}
```

- **נקודת בדיקה לתלמיד (Checkpoint):** מדוע אנו יוצרים את הטבלה בתוך `InitAsync()` הא-סינכרונית ולא בתוך הבנאי `public SqliteHeroService()`?

בלוק 4: החלפת `MockHeroService` ב-`SqliteHeroService` הודות לממשק `IHeroService`



- **קבצים ושכבה:**
  - `LearnSqlite/Views/HeroesListPage.xaml.cs`
  - `LearnSqlite/Views/AddHeroPage.xaml.cs`
  - `LearnSqlite/Views/HeroDetailsPage.xaml.cs`
- **מה הקוד עושה:** מאחר שכל שלושת ה-ViewModels (`HeroesListViewModel`, `AddHeroViewModel`, `HeroDetailsViewModel`) מקבלים בבנאי שלהם את הממשק `IHeroService` ולא מחלקה קונקרטית, **אין צורך לשנות אף שורת קוד ב-ViewModels או בקובצי ה-XAML!** כל מה שנדרש הוא להחליף את יצירת המופע מ-`new MockHeroService()` ל-`new SqliteHeroService()`:

CSHARP

```
//  ב-HeroesListPage.xaml.cs:
 _viewModel = new HeroesListViewModel(new SqliteHeroService());

 // ב-AddHeroPage.xaml.cs:
 BindingContext = new AddHeroViewModel(new SqliteHeroService());
```

```
// ב-HeroDetailsPage.xaml.cs:
BindingContext = new HeroDetailsViewModel(new SqliteHeroService());
```

- **נקודת בדיקה לתלמיד (Checkpoint):** כיצד עבודה מול הממשק `IHeroService` הקלה עלינו במעבר מנתוני דמה בזיכרון (`MockHeroService`) למסד נתונים אמיתי (`SqliteHeroService`), ואיזה שלב עתידי נוכל להוסיף ב-`MauiProgram.cs` (Dependency Injection) כדי שאפילו קובצי ה-Code-Behind לא יצטרכו להכיר את `SqliteHeroService`?